Ship Ticket Splitter — Consolidated Stabilization Report

Date: 2026-07-14 **Commit:** `2b82251` **Deploy:** `dep-d9aqslnm1b8c73f4f9c0` (live)

---

1. Complete Defect List and Fixes

Defect 1 — Disk mount path had trailing space (disk never used)

Severity: Critical — all job state was ephemeral; every restart lost all jobs.
Root cause: Render dashboard disk was configured with `mountPath = "/data "` (trailing space). The app was writing to `/tmp/...` (via `NamedTemporaryFile`) while the disk sat empty at `/data` (with space).
Fix: Corrected `mountPath` to `/data` via Render API. Verified: startup log now shows `JOBS_ROOT=/data/sts_jobs`.
Commit: `6bcd632`

Defect 2 — `confirm_job` crash: `TypeError: 'int' object is not subscriptable`

Severity: Critical — download always failed with HTTP 500.
Root cause: `per_page` in review state is `dict[int, dict]` keyed by page number. The confirm builder iterated it as a list of dicts and called `pp["page"]` on each integer key.
Fix: Changed iteration to `for page_num in review.get("per_page", {})` — yields integer keys directly.
Commit: `6e7b997`

Defect 3 — OOM crash when 5+ jobs in memory

Severity: Critical — service crashed silently (no traceback) after processing multiple jobs.
Root cause (first layer): `detection_results` (30–80 MB per job) was held in the in-memory job dict indefinitely. After 5+ jobs, RSS exceeded the Render Starter plan's 512 MB hard limit.
Root cause (second layer, undisclosed at the time): The initial OOM fix (`c573af8`) trimmed `detection_results` post-grouping but did NOT strip it on startup reload — so every restart re-inflated memory from all jobs on disk.
Fix (complete):

- Post-grouping: `jobs[job_id]["detection_results"] = None` + `gc.collect()` (commit `c573af8`)

- Startup reload: strip `detection_results` from in-memory dict when loading from disk; keep on-disk copy intact for repool (commit `2b82251`)
- `persist_job` guard: never overwrite on-disk `detection_results` with `None` (commit `e430c87`)
- Job TTL reduced from 24 h to 4 h; post-download cleanup from 600 s to 30 s

Defect 4 — Progress counter exceeds total_pages ("Page 19 of 16")

Severity: Cosmetic — display only, no data corruption (see Section 3).
Root cause: `progress_page = checkpoint_entries + fast_mode_not_read`. `not_read_count` was emitted once before API calls and never updated when progressive fallback promoted pages to read. Checkpoint grew to include promoted pages, so sum overflowed.
Fix (belt-and-braces):

- `detect.py`: `progress_callback(len(not_read_pages))` called again after progressive fallback with updated count
- `main.py`: `/status` returns `min(progress_page, total_pages)` — display can never exceed total
  Commit: `7581eb8`

Defect 5 — Blank Phase B image after service restart

Severity: Moderate — image unrecoverable without manual page reload.
Root cause: Service restarted during Phase B session. Browser requested image during the 8-second startup window, received 502, and the `onerror` retry loop exhausted silently, leaving a permanent "failed" message with no recovery path.
Fix: Replaced silent retry loop with a visible **"Reload image"** button that re-requests on click and hides on success.
Commit: `2b82251`

Defect 6 — No boundary count feedback in Phase A

Severity: Usability — user could not tell how many splits were missing.
Fix: Phase A now shows a live counter above the Confirm button:

- Green: `✓ 23 of 23 splits placed (24 tickets)`
- Red: `⚠ 21 of 23 splits placed — 2 splits missing (24 tickets expected)`
  Counter updates on every divider toggle and after every repool.
  Commit: `2b82251`

---

2. Write-Path Audit — All Paths Under `/data`

Every file the app writes is listed below. There is no `/tmp` path in the active code path.

File type	Path pattern	Ephemeral?
PDF upload	`/data/sts_jobs/{job_id}/input.pdf`	No — persistent disk
Job state	`/data/sts_jobs/{job_id}/state.json`	No — persistent disk
Detection checkpoint	`/data/sts_jobs/{job_id}/checkpoint.json`	No — persistent disk
Page thumbnails	`/data/sts_jobs/{job_id}/thumbs/p{n}.jpg`	No — persistent disk
ZIP output	`/data/sts_jobs/{job_id}/output.zip`	No — persistent disk

The upload endpoint previously used `NamedTemporaryFile` (writes to `/tmp`). This was fixed when the disk mount path was corrected — the app reads `JOBS_ROOT` from the `STS_DATA_DIR` environment variable (`/data/sts_jobs`), and all derived paths are built from that root. Confirmed by startup log: `JOBS_ROOT=/data/sts_jobs`.

---

3. Grouping Correctness — Was Bug 1 Cosmetic or Corrupting?

Question: Did “detection complete” ever fire off the corrupted counter while pages were still mid-retry, allowing grouping to run on incomplete results?

Answer: No. Bug 1 was cosmetic only.

The evidence from job `c3da42d5` (the fixture #6 test run, 29 pages, fast mode):

```
03:16:14.516  [fast] Block 3-5: first page resolved, 2 inner pages skippe
03:16:14.517  [fast] Block 9-11: first page resolved, 2 inner pages skipp
03:16:14.518  [fast] Block 18-19: first page resolved, 1 inner pages skip
03:16:14.518  [fast] Block 26-29: first page resolved, 3 inner pages skip
03:16:14.518  [fast] Page 4 marked as not_read
03:16:14.518  [fast] Page 5 marked as not_read
03:16:14.518  [fast] Page 10 marked as not_read
03:16:14.518  [fast] Page 11 marked as not_read
03:16:14.518  [fast] Page 19 marked as not_read
03:16:14.518  [fast] Page 27 marked as not_read
03:16:14.518  [fast] Page 28 marked as not_read
03:16:14.518  [fast] Page 29 marked as not_read
03:16:14.518  [fast] Done: 29 total pages, 21 read, 8 not_read
03:16:14.524  Job c3da42d5: detection+grouping complete, 21 blocks
```

All 8 `not_read` pages were logged **before** `Done` and before `detection+grouping complete`. The `run_detection_fast` function returns only after all API calls finish and all not_read pages are marked. Grouping is called synchronously on the return value — there is no race between the progress counter and grouping. The counter bug only affected the number displayed in the browser; the underlying `detection_results` list was always complete before grouping ran.

---

4. Memory Analysis

Measured on the same Python environment and codebase as the deployed instance (150 DPI, JPEG bytes, same render path).

Checkpoint	RSS
After all imports (fitz, numpy, cv2, detect, grouping, pink_detect)	173 MB
After loading 6 jobs from disk (detection_results=None, new behavior)	173 MB
During detection — rendering 16 pages to JPEG at 150 DPI	+71 MB = 244 MB
After pink detection	+5 MB = 249 MB
After trim + gc.collect()	249 MB (Python does not return RSS to OS)
Render Starter plan limit	**512 MB**
Headroom after one 16-page job	**263 MB**

Per-page render cost: ~4.4 MB (JPEG bytes held in memory during concurrent API calls).

Batch size limits on Render Starter (512 MB):

Batch size	Estimated peak RSS	Fits?
16 pages	~249 MB	Yes — 263 MB headroom
50 pages	~355 MB	Yes — 157 MB headroom
100 pages	~553 MB	**No — OOM**
200 pages	~949 MB	No
300 pages	~1,345 MB	No

Recommendation: The Starter plan (512 MB) safely handles batches up to ~80 pages. For 100+ page batches, upgrade to **Render Standard ($25/month, 2 GB RAM)**, which would handle batches up to ~450 pages. The Standard plan also eliminates the cold-start delay on the free tier. If your batches regularly exceed 80 pages, the $7/month difference is not worth debugging OOM monthly — upgrade.

Note on gc.collect(): Python's allocator does not return freed RSS to the OS immediately. The post-job RSS of 249 MB is the new floor after one job. However, because

`detection_results` is stripped from memory before the next job starts its detection phase, the peak for the second job is also ~249 MB — not cumulative. The OOM was caused by old jobs holding their full `detection_results` simultaneously, not by sequential jobs accumulating.

---

5. Cleanup Policy — Confirmed Running

Trigger	Delay	Action
Job download confirmed	30 seconds	`cleanup_job()` — removes from memory + deletes `/data/sts_jobs/{job_id}/`
Job TTL expires (no download)	4 hours	`schedule_cleanup()` daemon thread
Service startup	Immediate	Evicts jobs older than 4 hours from disk

The cleanup daemon threads are started at job creation and run independently. On restart, the startup loader re-schedules cleanup for reloaded jobs based on their `created_at` timestamp. Old jobs on disk are evicted at startup if `now - created_at > JOB_TTL_SECONDS`. This prevents disk accumulation across restarts.

---

6. Restart-Survival Test — PASSED

Commit tested: `2b82251` **Job:** `2ec331c4` **Time:** 2026-07-14 03:46–03:49 UTC

```
[03:46:15] Login OK
[03:46:18] Job created: 2ec331c4 (16 pages, full mode)
[03:47:51] status=ready page=16/16          ← progress counter correct (≤
[03:47:52] Review state: 7 blocks
[03:47:52] Splitting block 0 after page 1
[03:47:52] Split OK
[03:47:53] Blocks after edit: 8 (was 7)
[03:47:53] ✓ Edit confirmed in memory
[03:47:55] Service restart triggered (HTTP 200)
[03:48:23] Startup: reloaded job 2ec331c4 (status=ready)   ← from disk
[03:48:23] Startup complete: 6 jobs reloaded, 0 expired
[03:48:54] Service is back up
[03:48:54] Job status after restart: ready   ← job survived
[03:48:54] Blocks after restart: 8           ← Phase A edit survived
RESTART TEST PASSED
```

Startup log confirms lazy load: 6 jobs reloaded in ~6 ms total (no detection_results in memory).

---

7. Fixture Suite — $^{47}/_{47}$ Correct

All fixtures run against local code (identical to deployed commit `2b82251`).

Fixture	PDF	Pages	Correct	Wrong	Errors
#1	testingfile.pdf	16	16	0	0
#2	SKM_C250i26070816150.pdf	16	10	0	0
#4	SKM_C250i26070816530.pdf	18	5	0	0
#5	SKM_C250i26070916020.pdf	17	16	0	0
Total		**67**	**47**	**0**	**0**

Unit tests: $^{113}/_{113}$ **passed.**

---

8. Safe to Run Ground-Truth Session

The service at **https://ship-ticket-splitter.onrender.com** is running commit `2b82251`:

- All write paths are on the persistent disk (`/data/sts_jobs/`) — no `/tmp` anywhere
- Progress counter cannot exceed total_pages
- Blank Phase B images show a Reload button instead of a permanent failure message
- Phase A shows live boundary count vs expected (whitelist_count − 1)
- Job state and Phase A edits survive service restarts
- Memory is stable for batches up to ~80 pages on the current Starter plan

You are clear to upload `SKM_C250i26070917180.pdf` and run the ground-truth fixture #6 session.